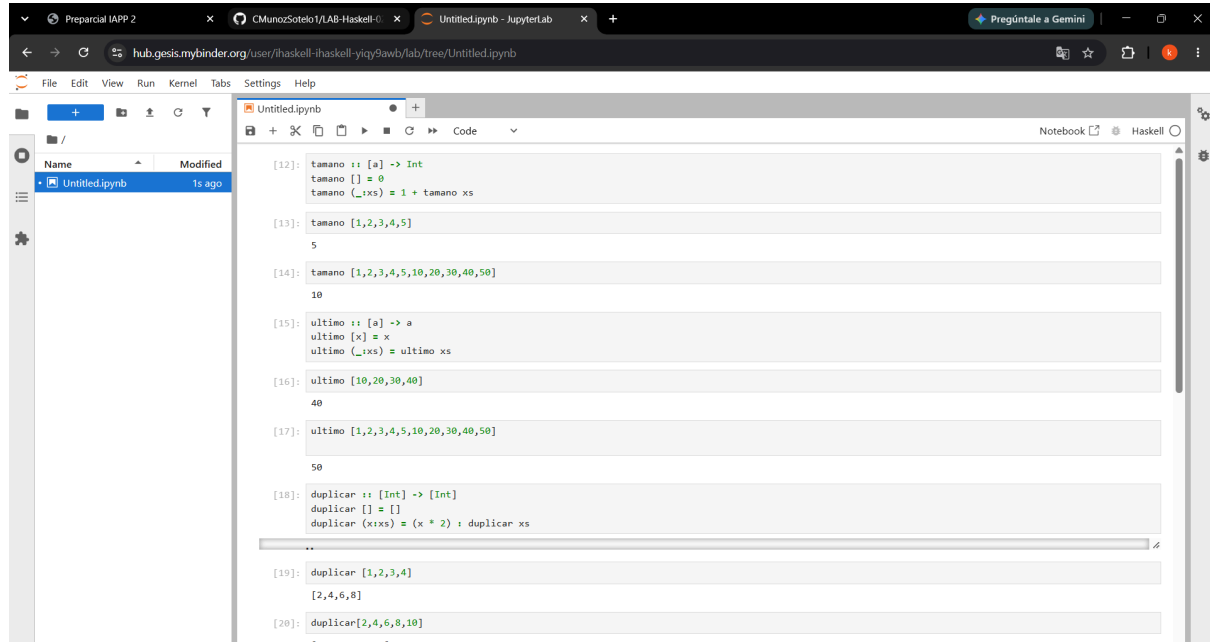


Camilo Andres Muñoz Sotelo

Punto 1

Definir las siguientes funciones utilizando recursión



```
[12]: tamaño :: [a] -> Int
      tamaño [] = 0
      tamaño (_:xs) = 1 + tamaño xs

[13]: tamaño [1,2,3,4,5]
      5

[14]: tamaño [1,2,3,4,5,10,20,30,40,50]
      10

[15]: ultimo :: [a] -> a
      ultimo [x] = x
      ultimo (_:xs) = ultimo xs

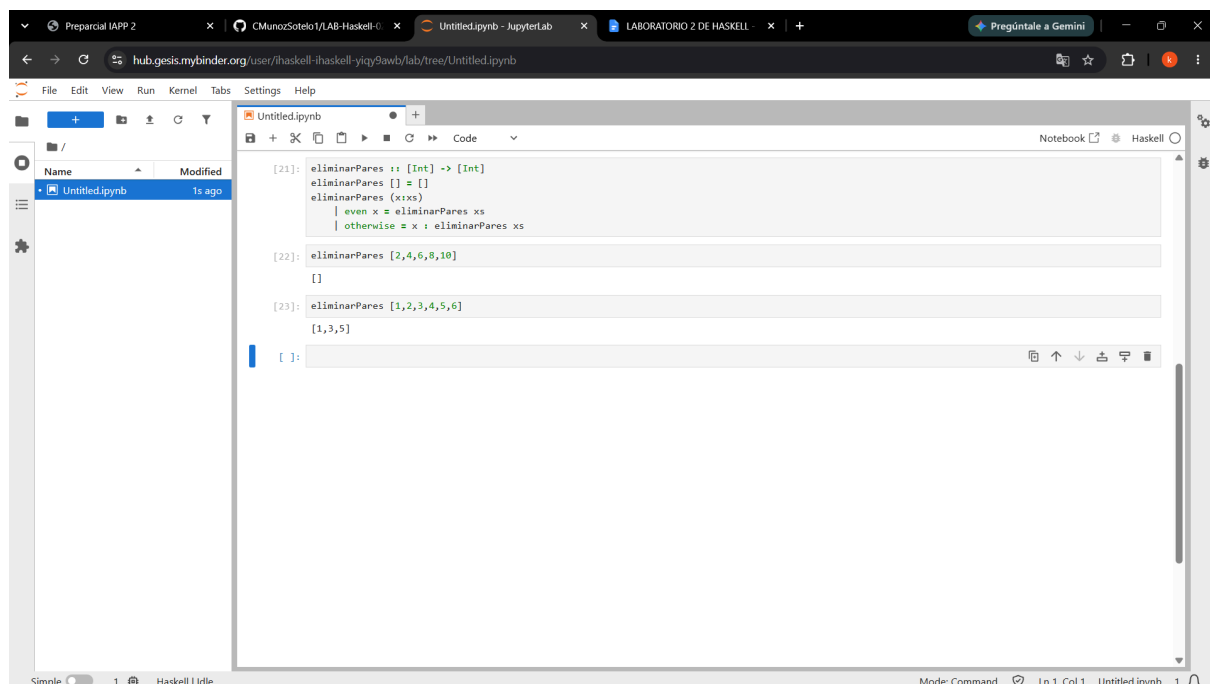
[16]: ultimo [10,20,30,40]
      40

[17]: ultimo [1,2,3,4,5,10,20,30,40,50]
      50

[18]: duplicar :: [Int] -> [Int]
      duplicar [] = []
      duplicar (x:xs) = (x * 2) : duplicar xs

[19]: duplicar [1,2,3,4]
      [2,4,6,8]

[20]: duplicar [2,4,6,8,10]
      [4,8,12,16,20]
```



```
[21]: eliminarPares :: [Int] -> [Int]
      eliminarPares [] = []
      eliminarPares (x:xs)
        | even x = eliminarPares xs
        | otherwise = x : eliminarPares xs

[22]: eliminarPares [2,4,6,8,10]
      []

[23]: eliminarPares [1,2,3,4,5,6]
      [1,3,5]

[ ]:
```

Define estas mismas funciones pero utilizando funciones de orden superior como map y filter

The screenshot shows a JupyterLab notebook with the following code:

```
[1]: duplicar2 :: [Int] -> [Int]
duplicar2 = map (*2)

[2]: duplicar2 [1,2,3,4]
[2,4,6,8]

[3]: duplicar2 [10,20,30,40]
[20,40,60,80]

[4]: eliminarPares2 :: [Int] -> [Int]
eliminarPares2 = filter odd

[5]: eliminarPares2 [1,2,3,4,5,6]
[1,3,5]

[6]: eliminarPares2 [10,20,30,40]
[]

[ ]:
```

Punto 2

1. Usa compresión de listas para
 - a. Obtener los cuadrados de los primeros 10 números.
 - b. Filtrar los divisibles por 3 entre 1 y 30.
 - c. Generar una lista de pares (x, y) donde $x < y$ y ambos estén en $[1..5]$.

The screenshot shows a JupyterLab notebook with the following code:

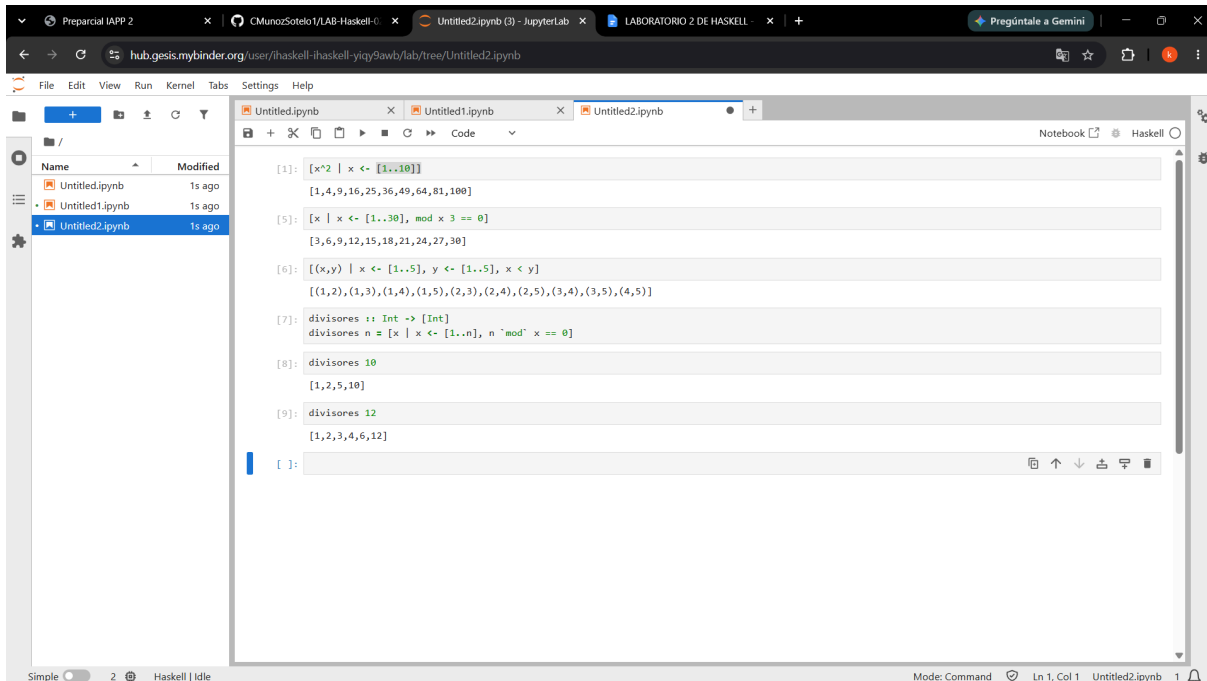
```
[1]: [x^2 | x <- [1..10]]
[1,4,9,16,25,36,49,64,81,100]

[5]: [x | x <- [1..30], mod x 3 == 0]
[3,6,9,12,15,18,21,24,27,30]

[6]: [(x,y) | x <- [1..5], y <- [1..5], x < y]
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

[ ]:
```

2. Define una función que devuelva los divisores de un número (usando lista por comprensión):
divisores :: Int -> [Int]



The screenshot shows a JupyterLab notebook with the following code cells:

```
[1]: [x^2 | x <- [1..10]]
[1,4,9,16,25,36,49,64,81,100]

[5]: [x | x <- [1..30], mod x 3 == 0]
[3,6,9,12,15,18,21,24,27,30]

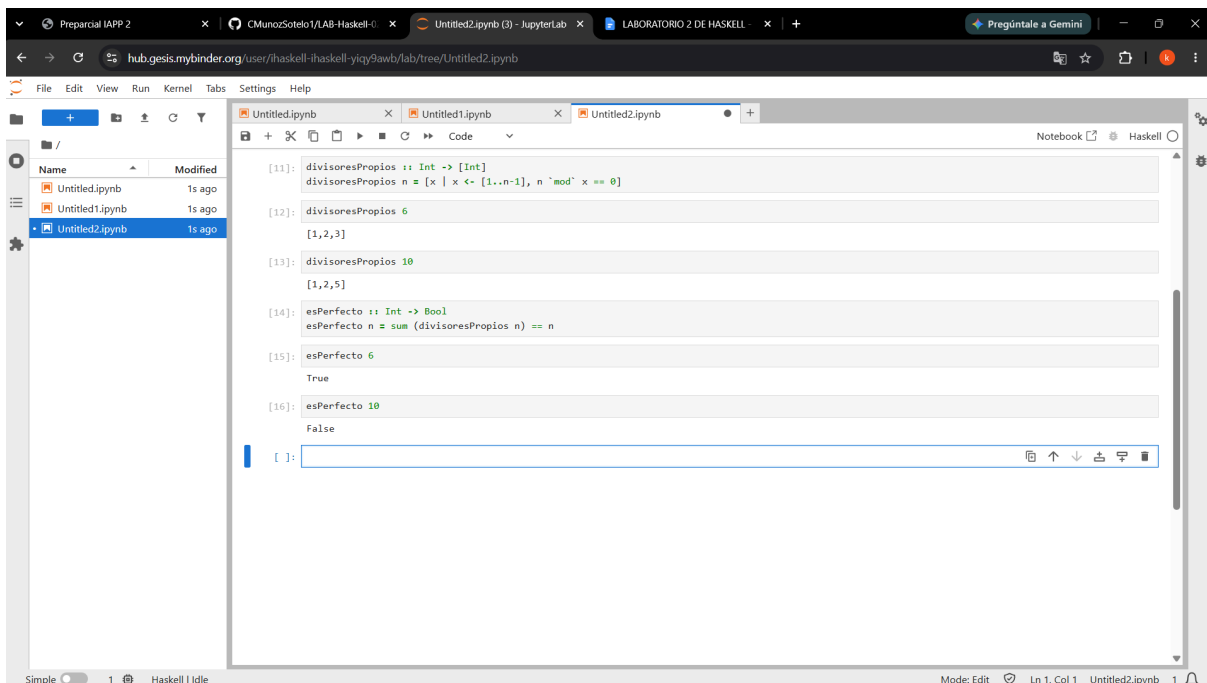
[6]: [(x,y) | x <- [1..5], y <- [1..5], x < y]
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

[7]: divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n `mod` x == 0]

[8]: divisores 10
[1,2,5,10]

[9]: divisores 12
[1,2,3,4,6,12]
```

3. Define la función divisores propios, crea una función que determine si un número es perfecto



The screenshot shows a JupyterLab notebook with the following code cells:

```
[11]: divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], n `mod` x == 0]

[12]: divisoresPropios 6
[1,2,3]

[13]: divisoresPropios 10
[1,2,5]

[14]: esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n

[15]: esPerfecto 6
True

[16]: esPerfecto 10
False
```

Punto 3

The screenshot shows a JupyterLab environment running in a web browser. The browser's address bar indicates the URL is `hub.gesis.mybinder.org/user/ihaskell-ihaskell-yiqy9awb/lab/tree/Untitled3.ipynb`. The JupyterLab interface includes a top toolbar with icons for file operations, a left sidebar with a file browser, and a main area with a code editor and a console.

In the file browser, a list of files is shown:

Name	Modified
Untitled.ipynb	1s ago
Untitled1.ipynb	1s ago
Untitled2.ipynb	9m ago
Untitled3.ipynb	1s ago

The code editor displays the following Haskell code:

```
[1]: sumalista :: [Int] -> Int
    sumalista xs = sum xs
```

The console shows the output of the execution:

```
[2]: sumalista [10,11,12]
    33
```

The status bar at the bottom indicates the current mode is 'Edit' and the file is 'Untitled3.ipynb'.